Visual-Inertial SLAM

Arihant Jain
Electrical and Computer Engineering
UC San Diego

***Abstract*—Building upon the previous project, we now tackle the Simultaneous Localization and Mapping(SLAM) problem, where we use advanced tools to perform accurate estimations of the world around us. We implemented Visual-Inertial SLAM using IMU measurements along with visual features from a stereo camera using an Extended Kalman Filter (EKF).**

***Index Terms*—IMU, Stereo camera, Mapping, Localization, Odometry, SLAM, EKF**

I. Introduction

When performing SLAM, we must consider not only noise from our sensors, but also non-linear motion in our system due to perturbations or construction. More sensors, along with filtering tools, enable better estimations, and thus better localization and mapping.

There exist many tools for accurate localization, and accurate odometry. However, combining both for the SLAM problem is where things get more complex. To tackle this problem, we use an Extended Kalman Filter (EKF), to ensure robustness to noise, nonlinearities in our system, and effective prediction and update of our state at each sensor measurement.

Our goal is to use sensor data from an IMU as well as a stereo camera to detect static features in the environment, and use them to build a map of the MIT campus as the robot drives through it. We will be applying our work onto 3 different datasets, and attempting feature extraction on our own and comparing against the existing features found.

II. Problem Formulation

We divide the problem into 4 tasks: IMU odometry prediction, Feature extraction from stereo camera images, visual mapping using features and odometry measurements, and finally Visual-Inertial SLAM using all sensor measurements.

A. IMU Odometry

Here, we use the basic pose motion model to compute the predicted pose at each timestep as follows:

$u_t = [v_t, \omega_t]^T \in \mathbb{R}^6$: Twist matrix at time t.

$\mu_{t|t} = \zeta_t \in \mathbb{R}^6$: twist in world frame at time t.

dt delta time between timestep $t-1$ and t.

$w \sim \mathcal{N}(0, W)$ measurement noise.

We can then use our twist matrix from the IMU and our current pose to calculate the next expected pose using the Extended Kalman Filter Prediction step. We repeat this over all timesteps to get our path taken.

For odometry, we are generally trying to minimize the difference between our predicted position and actual position: $\min_{x_{0:T}} \Sigma_0^{T-1} ||x_{t+1} - f(x_t, u_t, 0)||$. However, since we are only using the EKF prediction step, we will only calculate the following:

$$\mu_{t+1} = f(\mu_t, u_t, 0)$$

$$\Sigma_{t+1} = F_t^T \Sigma_t F_t + Q_t^T W Q_t$$

Where $F_t = df(\mu_t, u_t, 0)/dx$ and $Q_t = df(\mu_t, u_t, 0)/dw$

B. Landmark Mapping

For visual mapping, we take the robot pose from our motion model as assume it is accurate. We then compare our pose to features z found from our stereo camera, which all map to static landmarks m.

We can use the EKF update step to estimate the landmark locations m in the world frame, by triangulating the relative position of the features to the robot and using the robot pose to convert them into world coordinates.

We will be optimizing using the regular localization optimization function: $\min_m \Sigma_0^T ||z_t - h(x_t, 0)||$ for which we use a modified formulation of EKF Update step as will be shown in Section 3.

C. Visual-Inertial Odometry

We combine the problems from IMU odometry and visual mapping, and now perform the prediction and update step simultaneously for each timestep. Thus, this final step combines all our problems into one, involving the EKF prediction, update, and feature detection for camera images.

We will be optimizing over the cost of the entire SLAM problem, which is defined as follows:

$$\min_{x_{0:T}, m} (\Sigma_0^T ||z_t - h(x_t, 0)|| + \Sigma_0^{T-1} ||x_{t+1} - f(x_t, u_t, 0)||)$$

Here, we will use the whole EKF including prediction and update steps, and our implementation will be shown in Section 3.

III. Methods and Approach

A. IMU Odometry

Our predicted pose using the Extended Kalman Filter prediction step is as follows:

$$f(x_t, u_t, w) = x_t \exp(dt * \hat{u}_t) + w$$

Using this, we get the following:

$F = \exp(-dt * \widehat{u}_t)$, $Q = 1$

Finally, our EKF prediction step will be:

EKF Prediction Step

$\mu_{t+1} = \mu_{t+1|t} = \mu_t \exp(-dt * \hat{u}_t)$
$\Sigma_{t+1|t} = \exp(-dt * \widehat{u}_t)^T \Sigma_{t|t} \exp(-dt * \widehat{u}_t) + W$

where $\widehat{u}_t = [\hat{\omega}_t, \hat{v}_t; 0, \hat{\omega}_t]$ and $W \in \mathbb{R}^{6x6}$ is the noise covariance matrix, which is kept sufficiently small since we trust the IMU readings.

We can use our predicted twist and convert it to a pose matrix in the world frame $T_t \in \mathbb{R}^{4x4}$, which is then used for future steps.

B. Visual Landmark Mapping

For landmark mapping, we consider the poses from our IMU odometry as the ground truth, and use those to complete the update step for our landmarks.

Given our observed features $z_t \in \mathbb{R}^4$, we can triangulate the world frame position using the robot pose at time t, and the camera intrinsics. We initialize our landmarks to the first time the corresponding feature detects the landmark, rather than initializing to 0, to ensure more accurate mapping.

Our observation model is:

$$z_{t,i} = h(x_t, m, v) = K_s \pi({}_O T_I x_t^{-1} \underline{m_j}) + v$$

We use a mapping $\Delta(i) = j$ to map the observation i to the landmark j.

Visual Mapping

m - known landmark position
z - Observed camera feature
${}_O T_I$ - Pose from world to optical frame
Prior $m|z_{0:t} \tilde{} \mathcal{N}(\mu_t, \Sigma_t)$ with $\mu_t \in \mathbb{R}^{3M}$ and $\Sigma_t \in \mathbb{R}^{3Mx3M}$
EKF Update Step: Given an observation $z_{t+1} \in \mathbb{R}^{4N}$,
$K_{t+1} = \Sigma_t H_{t+1}^T (H_{t+1} \Sigma_t H_{t+1}^T + I \bigotimes V)^{-1}$
$\mu_{t+1} = \mu_t + K_{t+1}(z_{t+1} - K_s \pi({}_O T_I x_{t+1}^{-1} \mu_t)$
$\Sigma_{t+1} = (I + K_{t+1} H_{t+1}) \Sigma_t$
Where V is the observation noise, which we initialize to a relatively high value since we do not trust our observation as much as we trust our priors.

We run the EKF update step over each individual feature with an H matrix of size 4x3 instead of constructing a large matrix for all points, which we can then use to construct the Kalman gain for the specific feature, and update the part of our covariance matrix as well as position of that specific landmark.

We also implement a chi squared filter to ensure that the landmark position is not updated or initialized to be significantly far from the robot pose, and tune it for the specific data we are using.

C. Visual-Inertial SLAM

We will be combining the implementation from 3A and 3B, and also implementing the odometry update step using features detected at each step. We will use the motion and observatio models from the previous parts as well.

As in 3B, we use a mapping $\Delta(i) = j$ to map the observation i to the landmark j.

Visual-Inertial SLAM

Priors
$T_{t+1}|z_{0:t}, u_{0:t} \tilde{} \mathcal{N}(\mu_{t+1|t}, \Sigma_{t+1|t})$ with $\mu_{t+1|t} \in SE(3)$ and $\Sigma_{t+1|t} \in \mathbb{R}^{6x6}$
EKF Prediction Step:
Given an input u_t,
$\mu_{t+1} = \mu_{t+1|t} = \mu_t \exp(-dt * \hat{u}_t)$
$\Sigma_{t+1|t} = \exp(-dt * \widehat{u}_t)^T \Sigma_{t|t} \exp(-dt * \widehat{u}_t) + W$
EKF Update Step:
Given an observation $z_{t+1} \in \mathbb{R}^{4N}$,
Predicted observation:
$\tilde{z_{t+1,i}} = h(\mu_{t+1|t}, 0) = K_s \pi({}_O T_I x_t^{-1} \underline{m_j})$
Jacobian of the observation:
$H_{t+1,i} = -K_s d\pi/dq({}_o T_I \mu_t^{-1} \underline{m_j})_o T_I (\mu_t^{-1} \underline{m_j})^{\odot}$
H_{t+1} is a vertical stack of all $H_{t+1,i}$.
$K_{t+1} = \Sigma_{Zt} H_{t+1}^T (H_{t+1} \Sigma_t H_{t+1}^T + I \bigotimes V)^{-1}$

$\mu_{t+1} = \mu_t \exp((K_{t+1}(z_{t+1} - \tilde{z_{t+1}})^{\wedge})$
$\Sigma_{t+1} = (I - K_{t+1} H_{t+1}) \Sigma_t$
For this problem, we need to define very large matrices for our kalman gain and our update equations, so the challenge does not only rest in solving the problem, but in implementing it efficiently. For this purpose, we use the lil_matrix from the python library scipy, and take advantage of matrix vectorized operations in numpy.

IV. Results

We use three different datasets, and successfully obtained results for IMU Odometry, Landmark Mapping, and Visual-Intertial SLAM using pre-detected features for each dataset.

An interesting observation is that the IMU Odometry is extremely accurate, to the point where it is almost indistinguishable from the VIO output, due to the low noise from the IMU.

A. IMU Odometry

We can see how, in figures 1, 2, and 3, the EKF Prediction only model performs for mapping the path of the ClearPath Jackals on MIT's campus.

B. Landmark Mapping

In figures 4, 5, and 6, we can see how we localize all the features provided in each dataset using the EKF Update step, and using the IMU Odometry as a ground truth.

C. Visual-Inertial SLAM

In figures 7, 8, and 9, we can finally see the complete EKF taking place at each timestep, predicting the robot pose from IMU readings, and then updating both the pose as well as the landmark positions from our camera observations. Interestingly, this map looks very similar to our map using only IMU prediction. This is because of us using a very robust IMU, with small noise margins.

However, when comparing the landmark points, we can actually notice some differences in the points, with points being

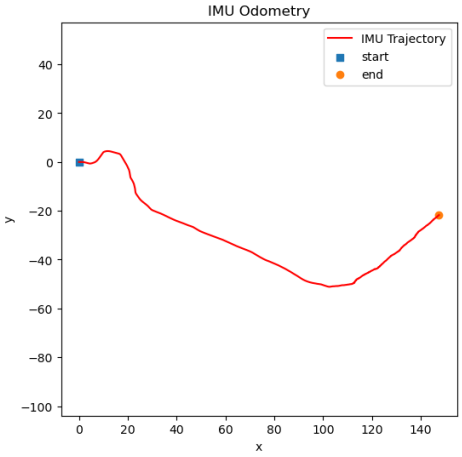


Fig. 1. IMU Odometry for dataset 00

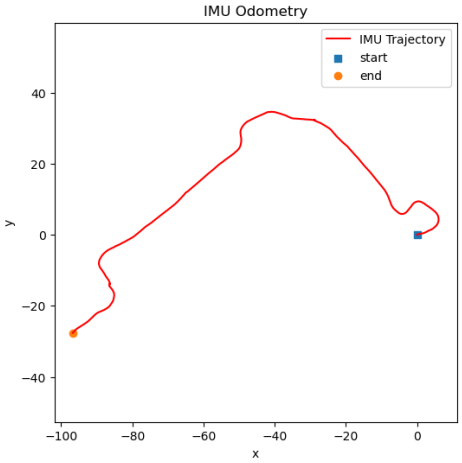


Fig. 2. IMU Odometry for dataset 01

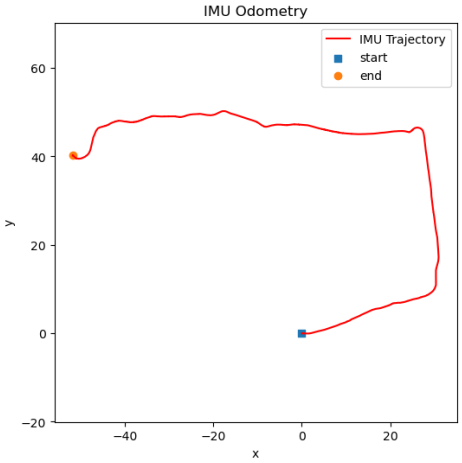


Fig. 3. IMU Odometry for dataset 02

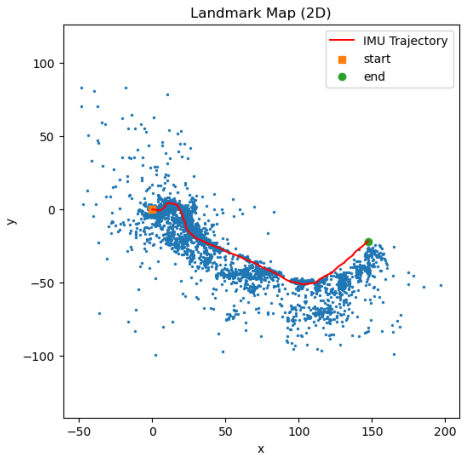


Fig. 4. Landmark Map for dataset 00

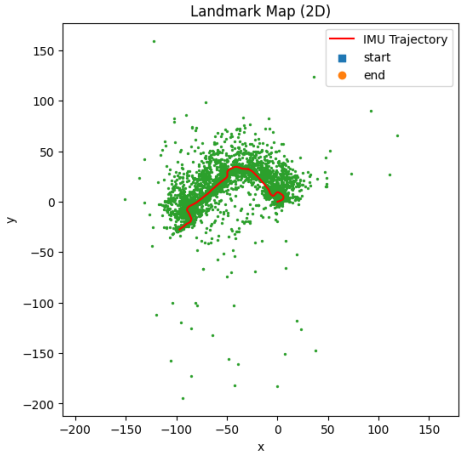


Fig. 5. Landmark Map for dataset 01

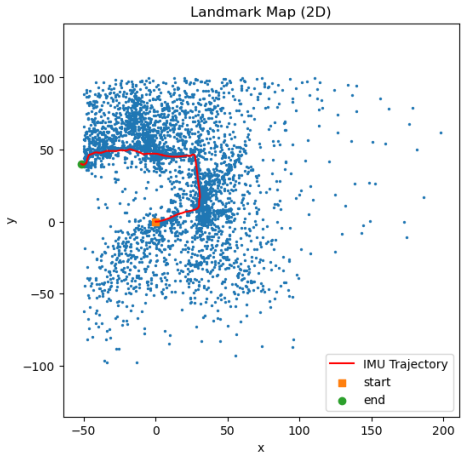


Fig. 6. Landmark Map for dataset 02

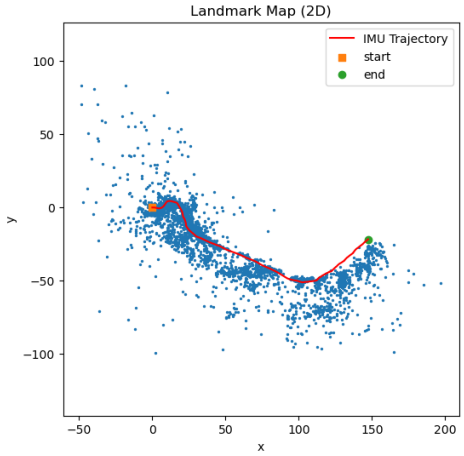


Fig. 7. VIO for dataset 00

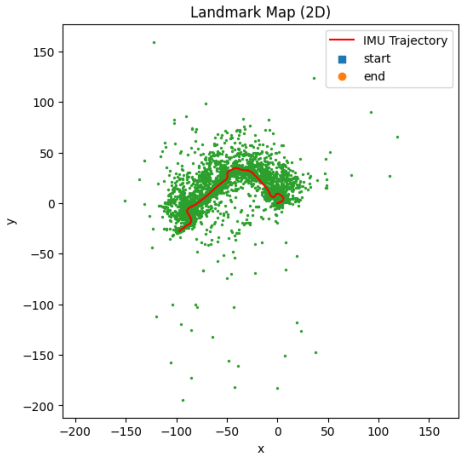


Fig. 8. VIO for dataset 01

spread wider than in the Update-only version of mapping. This could be due to a myriad of reasons, from improved filtering steps taken, to a better estimate of the landmarks being made when using the update and prediction step together.

V. Conclusion

In this report, we present a method of performing SLAM on a robot using a stereo camera and IMU measurements, effectively mapping landmark features detected from the camera and localizing using IMU measurements. We leverage a new estimations algorithm, the Extended Kalman Filter, to work with these non-linear systems involving projections and nonlinear motion.

We then apply this method on three datasets, comparing the performance of isolated odometry and mapping with the combined EKF SLAM solution. Our results appear promising and accurate, and illustrate that the combined EKF SLAM does not reduce accuracy, but actually can improve accuracy in these problems.

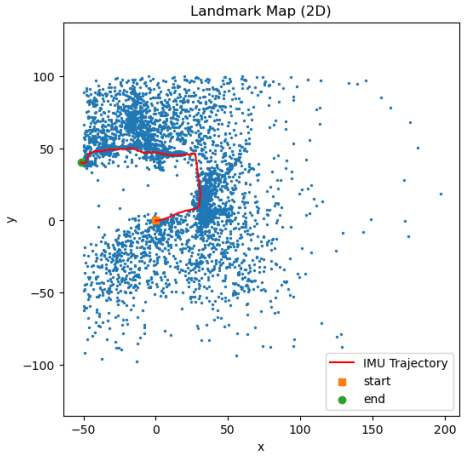


Fig. 9. VIO for dataset 02

References

1) Discussed at a high level the concepts of Optical Flow with ChatGPT (not implemented).
2) Used Prof Atanasov's class notes and links provided in the project description.